Compiler Construction – Spring 2026 – Final Exam Solution

Question#02: A lexical analyzer uses panic mode recovery. It skips characters until a **whitespace** (space, tab, newline) or a **delimiter** (;, ,, (,), {, }) is found.

Input string:
a = 10 @#$% b = 20 ; c = @30

(Marks: 4 + 5 + 5 + 6)

a) How many lexical errors occur?

Solution: Each **contiguous invalid sequence** is 1 lexical error (panic skips until whitespace/delimiter).

- Sequence 1: @#$% b
- Sequence 2: @30

So 2 Lexical Errors

b) For each error, state the position (character index) where recovery ends.

Solution:

Error 1: `@#$%` starts at position 8 and ends at position 11 (before space at 12).

Error 2: `@` (starts at position 26) → ends at position 28, at position 27, there 3, it is valid character but there is no space between @ and 3, but as per Panic Mode and rules gives in the question, there is no whitespace, delimiter etc, so it will be part of error.

c) What tokens are produced after recovery?
Tokens produced:

1. a (identifier)
2. = (operator)
3. 10 (number)
4. b (identifier)
5. = (operator)
6. 20 (number)
7. ; (delimiter)
8. c (identifier)
9. = (operator)

d) Suppose after recovery, the lexical analyzer inserts a special error token ERR instead of skipping tokens silently. Modify answers (a), (b), (c) accordingly, and explain how error recovery differs.

Solution:

What's Change: Instead of ignoring skipped chars, after each recovery, insert an ERR token.

Modified answers:

(a) How many lexical errors?
Same: **2** (still 2 panic recovery activations).

(b) Position where recovery ends per error:
Same: ends at 11, ends at 28 (recovery logic unchanged).

(c) Tokens produced:
Before error 1: a, =, 10
After error 1 recovery: insert ERR, then resume at pos 13: b, =, 20, ;, c, =
After error 2 recovery: insert ERR, then nothing else.

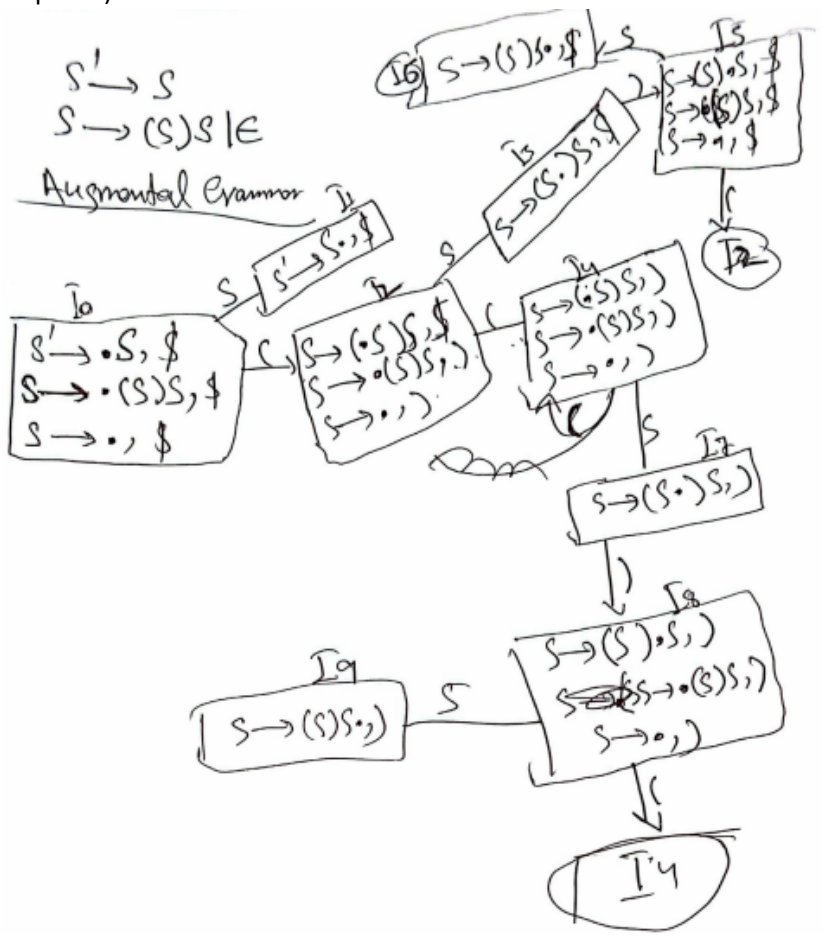
So tokens: a, =, 10, ERR, b, =, 20, ;, c, =, ERR

Question#03: For the given grammar, write the solution of following (Marks: 7 + 6 + 7)

S' → S

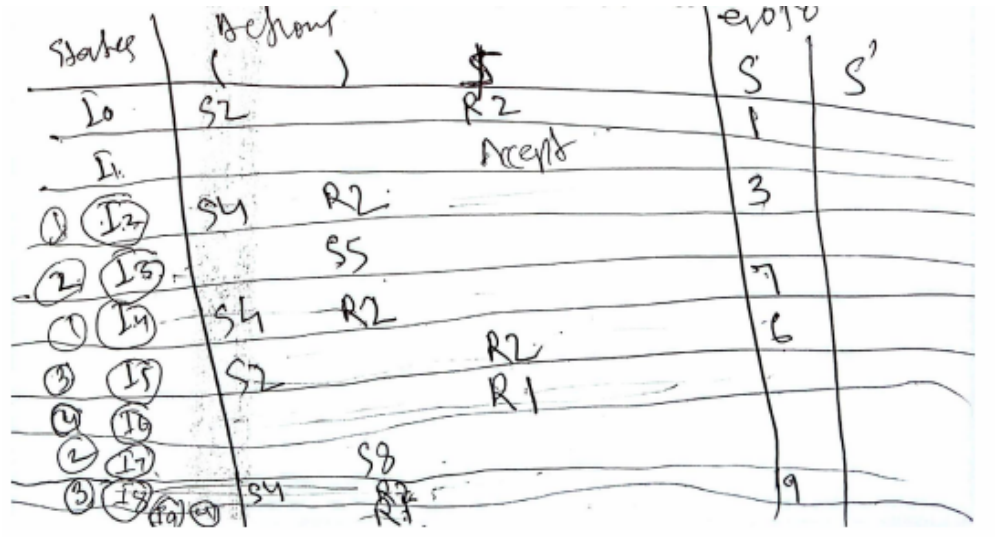
S → (S) S | ε

(a) Show the **LR(1) item sets** for this grammar, also mention the conflicts.(Parsing table is not required)

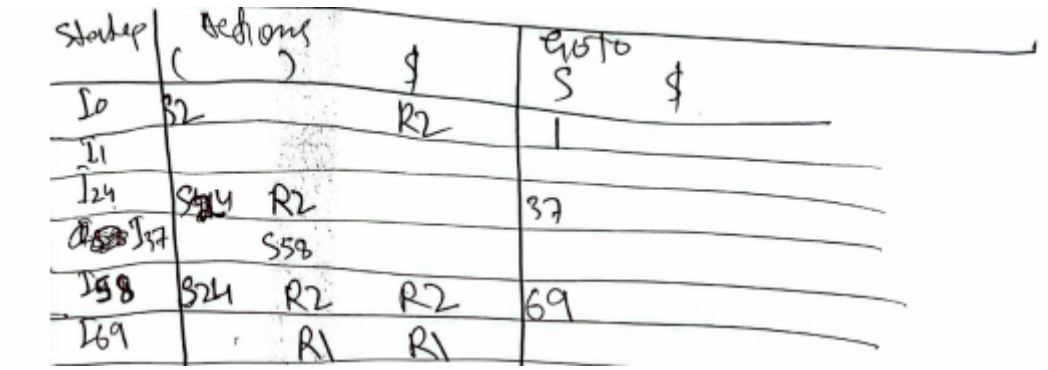


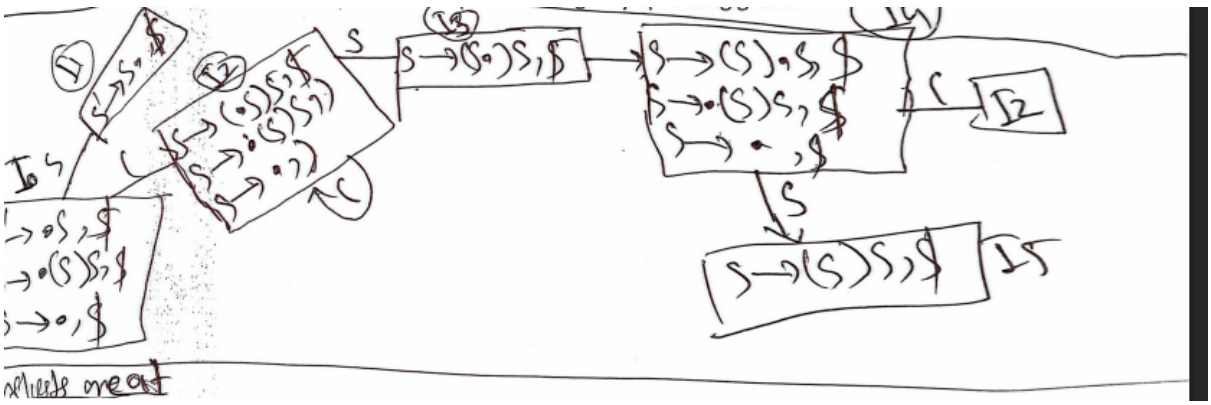
There is no any S/R or R/R Conflicts

(b) Merge all the **states** with identical cores but different lookaheads into LALR (Highlight merged states). also mention that will conflicts will remain same, increased or decreased.



Merged States





NO Conflicts also in LALR(1), Conflicts are same as LR(1)

(c) For a recursive descent parser, write the pseudocode for the production rule S.

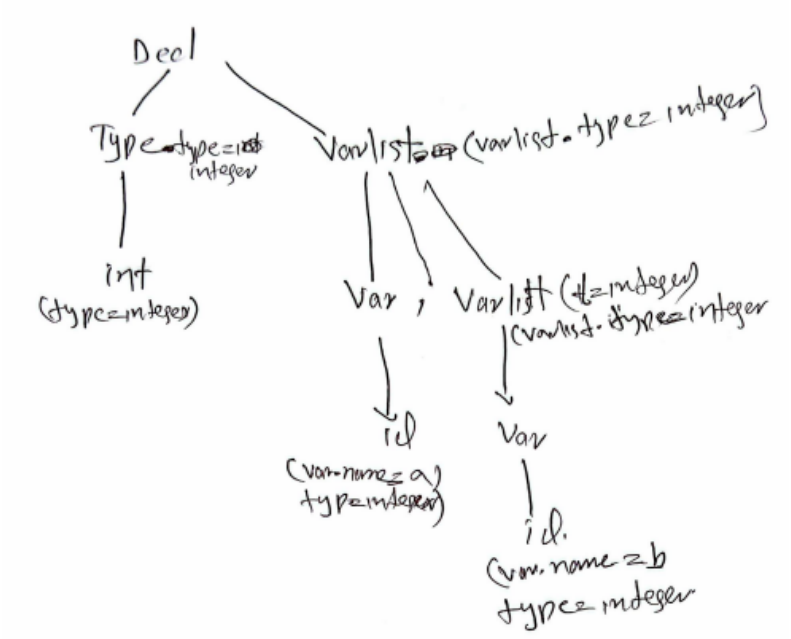
Solution:

```
procedure S()
   // Check if the lookahead matches the FIRST set of the first production
   if lookahead == '(' then
      match('(')
      S()
      match(')')
      S()
   else
      // Matches S -> ε
      // Do nothing and return successfully
      return
   end if
end procedure
```

Question #04: Given attribute grammar for a simple int declaration list:

Decl → Type VarList
Type → 'int' { Type.type = integer }
VarList → Var ',' VarList | Var
Var → 'id' { Var.name = id.lexeme;
Var.type = inherited from Type;
add_to_symbol_table(Var.name, Var.type); }
Synthesized attribute: none for Var, inherited attribute Type.type passes down.

a) Show the annotated parse tree for: int a, b;



b) Modify the grammar to also allow float declarations (float x, y;). Show the changed semantic rules

Solution:

Decl → Type VarList
Type → 'int' { Type.type = integer }
Type → 'float' { Type.type = float }
VarList → Var ',' VarList | Var
Var → 'id' { Var.name = id.lexeme;
Var.type = Type.type;
add_to_symbol_table(Var.name, Var.type); }

c) In the given grammar, is it possible to detect a redeclaration of the same variable a in the same declaration list (e.g., int a, a;)? If yes, at what point? If no, how would you modify the grammar/rules to detect it?

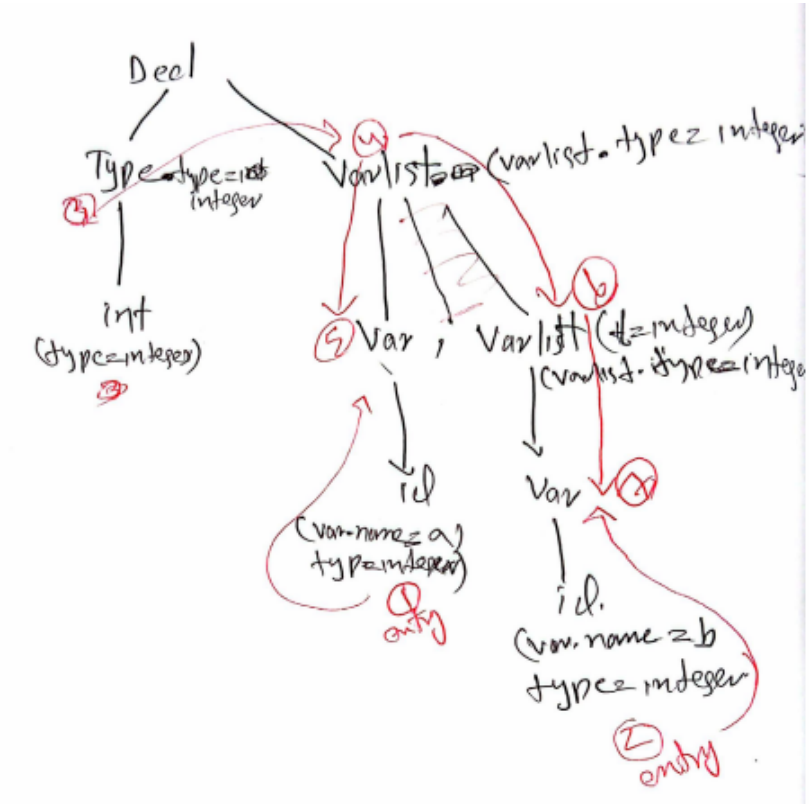
In the given grammar:
No, because add_to_symbol_table is called for each Var independently. No check prevents adding same name twice in same declaration.

Modification:

Change `VarList` rules to check:

```
VarList → Var ',' VarList
{
    if Var.name exists in current Decl's symbol table:
        error("redeclaration of " + Var.name);
    else:
        add_to_symbol_table(Var.name, Type.type);
}
VarList → Var
{
    same check as above
}
```

d) Draw the **dependency graph** for the parse tree of int **a, b;** using the original attribute grammar. Identify any cycles.

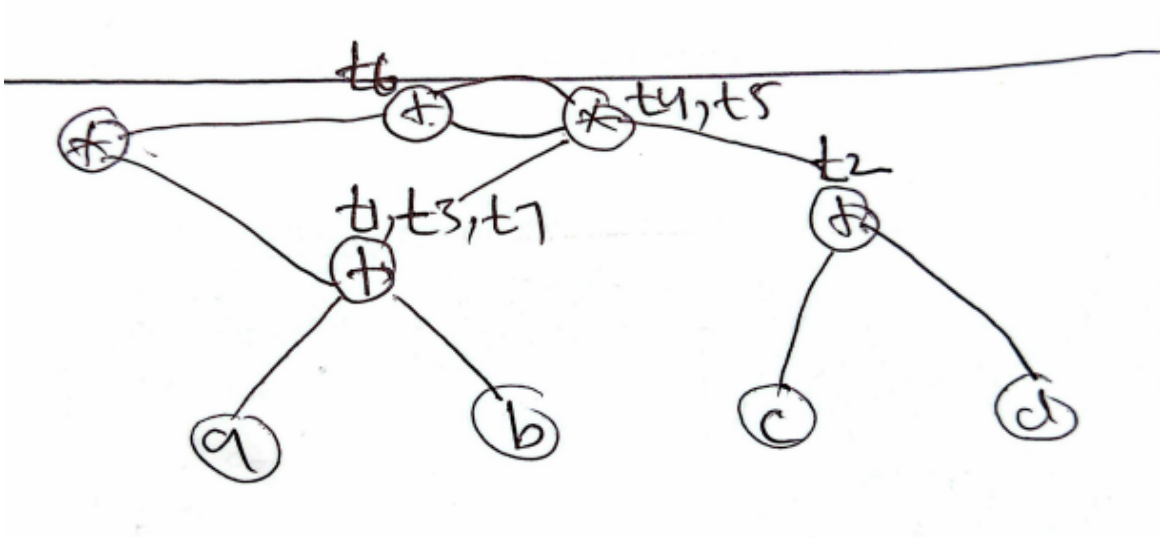


Question 05:

Consider the following basic block:

t1 = a + b
t2 = c + d
t3 = a + b
t4 = t1 * t2
t5 = t3 * t2
t6 = t4 + t5
t7 = a + b
t8 = t6 * t7

a) Draw the DAG for this basic block. Label each node with the operator and the variable(s) it defines.



b) Identify all common subexpressions in the block.

Solution:

Common subexpressions:

a + b (computed in t1, reused in t3 and t7)

(a + b) * (c + d) (computed in t4 and t5)

c) How many nodes are in the DAG? How many nodes would be in the corresponding AST?

The **DAG has 9 nodes (4 leaves: a, b, c, d; 5 internal nodes**: a+b, c+d, (a+b)*(c+d), (a+b)*(c+d) + (a+b)*(c+d), and that result multiplied by a+b).
The corresponding **AST has 16 nodes** (**8 leaf occurrences**: a three times, b three times, c once, d once; **8 internal nodes:** five + and three *).

d) Optimize the above block of code, also clearly state for each optimization which technique you have applied (e.g., "redundant load elimination", "algebraic simplification", "copy propagation", "common subexpression elimination via peephole", etc.) and finally write the optimized code.

Common Subexpression Elimination – a + b computed once as t1, reused for t3 and t7.

Common Subexpression Elimination – (a+b)*(c+d) computed once as t3, reused for t4 and t5 (originally t4 = t1*t2, t5 = t3*t2 with t3 = t1).

Algebraic Simplification – t4 + t5 where both are equal becomes 2 * t3 (i.e., t3 * 2).

Copy Propagation – After eliminating redundant t7 = a + b, use t1 directly in the final multiplication.

```
t1 = a + b

t2 = c + d

t4 = t1 * t2

t6 = 2 * t4      // because t5 = t4, so t4 + t5 = 2 * t4

t8 = t6 * s1
```